



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251233
Nama Lengkap	Mikael Gratianus Satrio Adi Kuncoro
Minggu ke / Materi	15 / Fungsi Rekursif

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

Pengertian Rekursif

Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri secara berulang hingga kondisi berhenti terpenuhi. Rekursif harus digunakan dengan hati-hati agar tidak menyebabkan perulangan tanpa batas (infinite loop) yang dapat membuat program hang. Fungsi rekursif memiliki dua bagian utama, yaitu **Base Case** sebagai kondisi berhenti dan **Recursive Case** sebagai bagian yang terus memanggil dirinya sendiri hingga mencapai Base Case.

Kelebihan dan Kekurangan

Kelebihan fungsi rekursif:

1. Kode program lebih singkat dan elegan.
2. Masalah kompleks dapat dipecah menjadi submasalah yang lebih kecil.
3. Cocok digunakan pada struktur data bertingkat seperti pohon (tree) atau grafik.
4. Membantu menyederhanakan logika penyelesaian masalah yang berulang atau berpola.
5. Membuat algoritma tertentu lebih mudah dipahami dibanding perulangan biasa.

Kekurangan fungsi rekursif:

1. Memakan memori lebih besar karena setiap pemanggilan membutuhkan ruang tambahan.
2. Mengorbankan efisiensi dan kecepatan eksekusi.
3. Sulit dilakukan debugging dan terkadang lebih sulit dipahami.
4. Berisiko menyebabkan infinite loop atau stack overflow jika Base Case tidak tepat.
5. Tidak selalu lebih efisien dibanding perulangan (looping) untuk masalah sederhana.

Bentuk Umum dan Studi Kasus

```
1  def function_name(parameter_list):  
2      ...  
3      function_name(...)  
4      ...
```

Gambar 1.1: Bentuk umum fungsi rekursif pada Python

Fungsi rekursif selalu memiliki solusi iteratif (perulangan). Salah satu contohnya adalah perhitungan faktorial. Faktorial merupakan operasi matematika untuk menghitung hasil perkalian berurutan dari angka 1 sampai n , yang ditulis sebagai $n!$. misalnya, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. Selain menggunakan rekursif, faktorial juga dapat dihitung dengan pendekatan iteratif menggunakan perulangan (looping), sehingga proses perhitungan dilakukan secara bertahap tanpa memanggil fungsi itu sendiri.

Algoritma menghitung faktorial secara iteratif:

1. Meminta input nilai n dari pengguna.
2. Menyiapkan variabel **total** untuk menyimpan hasil perkalian dan memberikan nilai awal **1**.
3. Melakukan perulangan dari $i = 1$ hingga n .
4. Pada setiap iterasi, nilai **total** dikalikan dengan i (**total** = **total** $\times$ i).
5. Setelah perulangan selesai, tampilkan hasil **total** sebagai nilai faktorial.

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{if } n > 0 \end{cases}$$

Gambar 1.2.: Rumus faktorial

Dari rumus pada Gambar 1.2. bisa dibuat pseudocode secara rekursif, contoh nya sebagai berikut:

Pseudocode (recursive):

function factorial is:

input: integer n such that $n \geq 0$

output: $[n \times (n-1) \times (n-2) \times \dots \times 1]$

1. if n is 0, return 1

2. otherwise, return $[n \times \text{factorial}(n-1)]$

end factorial

Gambar 1.3: Pseudocode rekursif faktorial

```
1  def faktorial(n):
3      return 1
4  else:
5      return faktorial(n-1) * n
6  print(faktorial(4))
```

Gambar 1.4: Source code rekursif faktorial dalam python

Hasil akhir nya adalah 24.

Berikut adalah gamabaran proses perhitungannya terjadi:

```
1.
2. calc_factorial(4)           # 1st call with 4
3. 4 * calc_factorial(3)       # 2nd call with 3
4. 4 * 3 * calc_factorial(2)   # 3rd call with 2
5. 4 * 3 * 2 * calc_factorial(1) # 4th call with 1
6. 4 * 3 * 2 * 1              # return from 4th call as number=1
7. 4 * 3 * 2                  # return from 3rd call
8. 4 * 6                      # return from 2nd call
9. 24                         # return from 1st call
```

Gambar 1.5: Proses perhitungan faktorial rekursif

Berikut studi kasus lainnya penggunaan rekursif dalam python:

Deret fibonacci

Deret Fibonacci adalah barisan bilangan di mana setiap angka merupakan penjumlahan dari dua angka sebelumnya. Barisan ini dimulai dari angka 0 dan 1. Contoh barisan Fibonacci awal: 0, 1, 1, 2, 3, 5, 8, 13, 21, dan seterusnya. Sama seperti faktorial, pencarian suku ke-n pada deret Fibonacci dapat diselesaikan dengan pendekatan iteratif maupun rekursif.

Algoritma menghitung Fibonacci secara iteratif:

- Meminta input suku ke-n dari pengguna.
- Jika $n = 0$ atau $n = 1$, langsung kembalikan nilai n .
- Jika $n > 1$, siapkan dua variabel awal (misal $a = 0$ dan $b = 1$) untuk menyimpan dua suku sebelumnya.
- Lakukan perulangan dari suku ke-2 hingga ke-n.
- Di setiap iterasi, hitung suku baru dengan menjumlahkan a dan b , lalu perbarui nilai a dan b untuk iterasi berikutnya.
- Setelah perulangan selesai, tampilkan hasil suku ke-n.

Fibonacci Sequence Formula

$$F_n = F_{n-1} + F_{n-2}, \text{ for } n > 2$$

here,

- F_n is the n^{th} term number,
- F_{n-1} is the $(n-1)^{\text{th}}$ term number, and
- F_{n-2} is the $(n-2)^{\text{th}}$ term number

Gambar 1.6: Rumus fibonacci

Dari rumus pada Gambar 1.4. bisa dibuat pseudocode secara rekursif, contoh nya sebagai berikut:

```

FUNCTION fibonacci(n)
    IF n == 0 THEN
        RETURN 0
    ELSE IF n == 1 THEN
        RETURN 1
    ELSE
        RETURN fibonacci(n - 1) + fibonacci(n - 2)
    ENDIF
ENDFUNCTION

```

Gambar 1.7: Pseudocode rekursif fibonacci

```

1  def fibonacci(n):
2      # Base Case
3      if n == 0:
4          return 0
5      elif n == 1:
6          return 1
7      # Recursive Case
8      else:
9          return fibonacci(n - 1) + fibonacci(n - 2)
10
11 # Menampilkan suku ke-5
12 hasil = fibonacci(5)
13 print(f"Suku ke-5 dari deret Fibonacci adalah: {hasil}")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\rekursif.py"
Suku ke-5 dari deret Fibonacci adalah: 5
PS D:\New folder - Copy\Coding>

```

Gambar 1.8: Source code rekursif fibonacci dalam python

Hasil akhir atau output nya adalah 5.

LINK GITHUB

: <https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git>

SOAL 01

```
1 def prima(n, i = None):
2     if n <= 1:
3         return False
4     if i == None:
5         i = n - 1
6     if i == 1:
7         return True
8     if n % i == 0:
9         return False
10
11     return prima(n, i-1)
12
13 inputan = int(input("Masukin bilangannya boss: "))
14 if prima(inputan):
15     print(f"{inputan} adalah bilangan prima")
16 else:
17     print(f"{inputan} bukan bilangan prima")
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal01.py"

Masukin bilangannya boss: 7

7 adalah bilangan prima

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal01.py"

Masukin bilangannya boss: 20

20 bukan bilangan prima

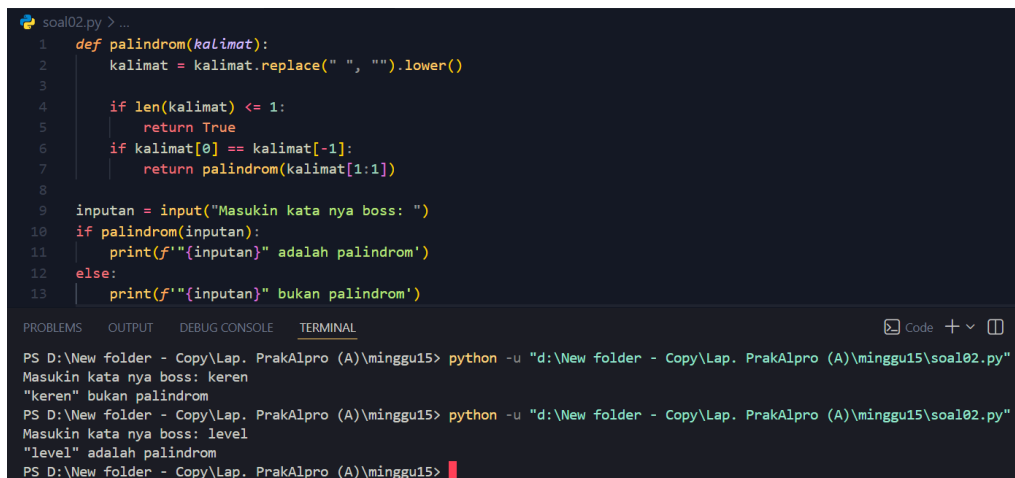
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15>

Gambar 1.9: Source code soal 01

Penjelasan:

Ini adalah program untuk menentukan suatu bilangan adalah bilangan prima atau bukan dengan cara rekursif. Pertama Fungsi **prima(n, i=None)** digunakan untuk mengecek apakah suatu angka termasuk bilangan prima dengan cara rekursif (fungsi memanggil dirinya sendiri). Jika angka **n** kurang dari atau sama dengan **1**, akan mengembalikan **False** karena bukan bilangan prima. Saat pertama kali dijalankan, **i** diisi dengan **n - 1** untuk mengecek pembagi dari angka terbesar menuju 2. Jika ditemukan **n** habis dibagi **i**, maka fungsi mengembalikan **False** karena angka tersebut memiliki pembagi lain selain 1 dan dirinya sendiri. Jika pengecekan terus berjalan sampai **i** menjadi **1** tanpa menemukan pembagi, fungsi mengembalikan **True**, artinya angka tersebut adalah bilangan prima. Pada program utama, pengguna diminta memasukkan angka, lalu fungsi **prima()** digunakan untuk mengecek hasilnya. Jika **True**, program menampilkan bahwa angka tersebut bilangan prima, sedangkan jika **False**, program menampilkan bahwa angka tersebut bukan bilangan prima.

SOAL 02



```
soal02.py > ...
1 def palindrom(kalimat):
2     kalimat = kalimat.replace(" ", "").lower()
3
4     if len(kalimat) <= 1:
5         return True
6     if kalimat[0] == kalimat[-1]:
7         return palindrom(kalimat[1:-1])
8
9     inputan = input("Masukin kata nya boss: ")
10    if palindrom(inputan):
11        print(f"{inputan}" adalah palindrom')
12    else:
13        print(f"{inputan}" bukan palindrom')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal02.py"
Masukin kata nya boss: keren
"keren" bukan palindrom
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal02.py"
Masukin kata nya boss: level
"level" adalah palindrom
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15>
```

Gambar 2.0: Source code soal 02

Penjelasan:

Ini adalah program untuk mengecek apakah suatu kata atau kalimat termasuk palindrom atau tidak. Pertama fungsi **palindrom(kalimat)** digunakan untuk mengecek apakah sebuah kata atau kalimat termasuk palindrom dengan cara rekursif. Pada awal fungsi, string dibersihkan menggunakan **.replace(" ", "")** untuk menghapus spasi dan **.lower()** untuk mengubah semua huruf menjadi kecil agar pengecekan lebih mudah. Jika panjang string tinggal **0** atau **1** karakter, fungsi langsung mengembalikan **True** karena berarti semua huruf sudah cocok (base case). Lalu, fungsi membandingkan huruf pertama dan terakhir, jika sama, kedua huruf tersebut dipotong lalu fungsi memanggil dirinya sendiri untuk mengecek bagian tengah string (**kalimat[1:-1]**). Jika huruf pertama dan terakhir tidak sama, fungsi otomatis menghasilkan **False** (None) sehingga kata dianggap bukan palindrom. Pada program utama, pengguna diminta memasukkan kata atau kalimat, lalu fungsi **palindrom()** digunakan untuk mengeceknya. Jika hasilnya **True**, program menampilkan bahwa input tersebut adalah palindrom, sedangkan jika **False**, program menampilkan bahwa input tersebut bukan palindrom.

SOAL 03



```
soal03.py > ...
1  def deret_ganjil(n):
2      if n <= 0:
3          return 0
4
5      if n % 2 != 0:
6          sisa = deret_ganjil(n-2)
7          if sisa == 0:
8              print(f"{n}", end="")
9          else:
10             print(f" + {n}", end="")
11             return n + sisa
12     else:
13         return deret_ganjil(n - 1)
14
15     inputan = int(input("Masukan angka nya boss: "))
16     print(f"Deret Ganjil: ", end="")
17     hasil = deret_ganjil(inputan)
18     print(f" = {hasil}")

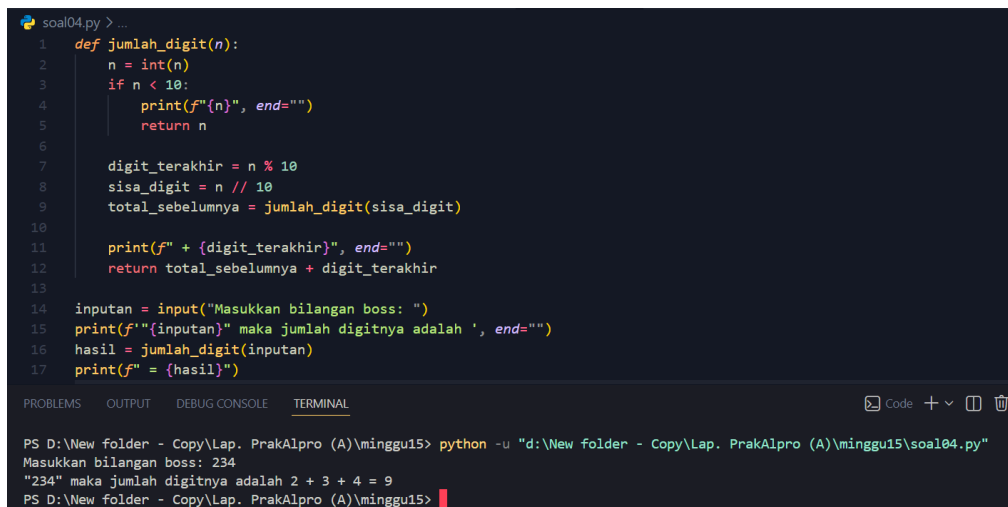
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal03.py"
Masukan angka nya boss: 15
Deret Ganjil: 1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 = 64
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15>
```

Gambar 2.1: Source code soal 03

Penjelasan:

Ini adalah program untuk menghitung jumlah deret ganjil dengan cara rekursif. Pertama fungsi **deret_ganjil(n)** digunakan untuk menampilkan dan menjumlahkan deret bilangan ganjil dari 1 sampai n dengan cara rekursif. Jika **n <= 0**, fungsi langsung mengembalikan **0** sebagai kondisi berhenti (base case). Lalu, fungsi mengecek apakah n adalah bilangan ganjil menggunakan **n % 2 != 0**. Jika ganjil, fungsi memanggil dirinya sendiri dengan **n-2** untuk mencari bilangan ganjil sebelumnya dan menyimpan hasilnya di variabel **sisa**. Setelah itu, angka dicetak, jika **sisa == 0**, angka dicetak tanpa tanda + karena merupakan angka pertama, sedangkan jika bukan, angka dicetak dengan tambahan + di depannya. **end=""** digunakan agar semua hasil tampil dalam satu baris. Fungsi kemudian mengembalikan hasil penjumlahan **n + sisa**. Jika n genap, fungsi langsung memanggil **deret_ganjil(n-1)** untuk mencari bilangan ganjil di bawahnya. Lalu, pengguna diminta memasukkan angka, lalu program mencetak label **"Deret Ganjil:"**, memanggil fungsi **deret_ganjil()** untuk menampilkan deret dan menghitung jumlahnya, kemudian hasil akhir penjumlahan ditampilkan pada terminal.

SOAL 04



```
soal04.py > ...
1 def jumlah_digit(n):
2     n = int(n)
3     if n < 10:
4         print(f"{n}", end="")
5         return n
6
7     digit_terakhir = n % 10
8     sisa_digit = n // 10
9     total_sebelumnya = jumlah_digit(sisa_digit)
10
11    print(f" + {digit_terakhir}", end="")
12    return total_sebelumnya + digit_terakhir
13
14    inputan = input("Masukkan bilangan boss: ")
15    print(f'"{inputan}" maka jumlah digitnya adalah ', end="")
16    hasil = jumlah_digit(inputan)
17    print(f" = {hasil}")

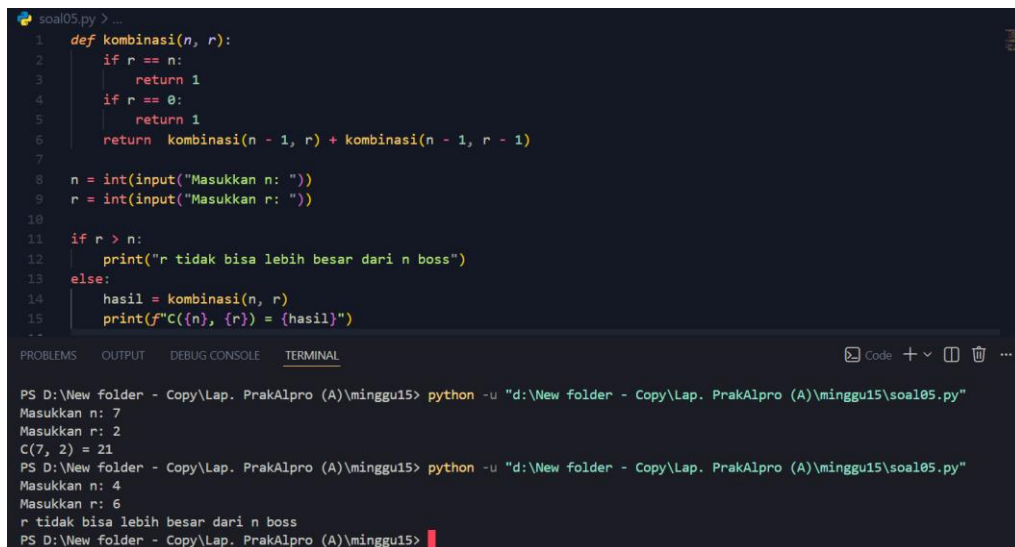
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal04.py"
Masukkan bilangan boss: 234
"234" maka jumlah digitnya adalah 2 + 3 + 4 = 9
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15>
```

Gambar 2.2: Source code soal 04

Penjelasan:

Ini adalah program untuk mengetahui jumlah digit dari suatu bilangan dengan cara rekursif. Pertama fungsi **jumlah_digit(n)** digunakan untuk menjumlahkan setiap digit dari sebuah bilangan dengan cara rekursif. Di awal fungsi, **n** diubah menjadi integer karena input dari pengguna biasanya berupa string. Jika **n** kurang dari **10** (hanya satu digit), fungsi langsung mencetak angka tersebut dan mengembalikan nilainya sebagai kondisi berhenti (base case). Jika lebih dari satu digit, fungsi mengambil digit terakhir menggunakan **n % 10** dan menyimpan sisa angkanya menggunakan **n // 10**. Selanjutnya, fungsi memanggil dirinya sendiri (**jumlah_digit(sisa_digit)**) untuk memproses digit sebelumnya. Setelah rekursi selesai, digit terakhir dicetak dengan tanda **+** di depannya agar terlihat seperti proses penjumlahannya, lalu fungsi mengembalikan hasil penjumlahan digit sebelumnya ditambah digit terakhir saat ini. Di program utama, pengguna diminta memasukkan bilangan, kemudian program mencetak teks awal, memanggil fungsi **jumlah_digit()** untuk menjumlahkan digit, menyimpan hasilnya ke variabel **hasil**, dan menampilkan total penjumlahannya di layar terminal.

SOAL 05



```
soal05.py > ...
1 def kombinasi(n, r):
2     if r == n:
3         return 1
4     if r == 0:
5         return 1
6     return kombinasi(n - 1, r) + kombinasi(n - 1, r - 1)
7
8 n = int(input("Masukkan n: "))
9 r = int(input("Masukkan r: "))
10
11 if r > n:
12     print("r tidak bisa lebih besar dari n boss")
13 else:
14     hasil = kombinasi(n, r)
15     print(f"C({n}, {r}) = {hasil}")
--

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Code + - - -

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal05.py"
Masukkan n: 7
Masukkan r: 2
C(7, 2) = 21
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu15\soal05.py"
Masukkan n: 4
Masukkan r: 6
r tidak bisa lebih besar dari n boss
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu15>
```

Gambar 2.3: Source code soal 05

Penjelasan:

Ini adalah program untuk menghitung kombinasi dengan cara rekursif. Pertama fungsi **kombinasi(n, r)** digunakan untuk menghitung kombinasi **C(n, r)**, yaitu jumlah cara memilih **r** item dari **n** item tanpa memperhatikan urutan dengan metode rekursif. Fungsi memiliki dua kondisi berhenti (base case), yaitu jika **r == n** maka fungsi mengembalikan **1** karena memilih semua item hanya memiliki satu kemungkinan, dan jika **r == 0** fungsi juga mengembalikan **1** karena tidak memilih item apa pun tetap dianggap satu cara. Jika tidak memenuhi kedua kondisi tersebut, fungsi menggunakan rumus rekursif **C(n, r) = C(n-1, r) + C(n-1, r-1)**, di mana **kombinasi(n-1, r)** berarti item terakhir tidak dipilih dan **kombinasi(n-1, r-1)** berarti item terakhir dipilih. Di program utama, pengguna diminta memasukkan nilai **n** dan **r**, lalu program memeriksa apakah **r > n**. Jika iya, program menampilkan pesan bahwa **r** tidak boleh lebih besar dari **n**. Jika valid, fungsi **kombinasi()** dipanggil untuk menghitung hasil kombinasi, kemudian hasilnya ditampilkan dalam format **C(n, r) = hasil**.